

— PROGRAMMERZAMANNOW · STUDI KASUS ARSITEKTUR

Senior Engineer Wajib Tahu CQRS

Search 3 detik → 100× lebih cepat

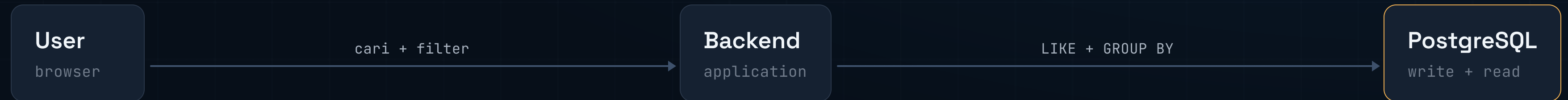
Command Query Responsibility Segregation —
memisahkan model **tulis (Command)** dari model
baca (Query) agar tiap sisi bisa dioptimalkan
independen.

● Studi kasus: pencarian katalog produk e-commerce · PostgreSQL → OpenSearch

Pencarian katalog produk

- Aplikasi punya **katalog produk** — produk, kategori, dan brand.
 - Data awalnya sedikit, jadi **semua terasa cepat**. Masalah baru muncul saat data membesar.
- User mengharapkan pencarian ala toko online
 - **Keyword search** — "sepatu", "wireless mouse"
 - **Filter** — kategori, brand, rentang harga, stok
 - **Facet** — jumlah produk per pilihan: Nike (20), Adidas (30)...
 - **Sorting & pagination**

Satu database saja



• Keyword

```
WHERE LOWER(name) LIKE '%kata%' OR LOWER(description)
LIKE '%kata%'
```

• Facet

Beberapa query **GROUP BY** terpisah — satu per kategori, brand, rentang harga, dan ketersediaan stok.

Arsitektur paling sederhana dan paling umum. Tidak ada yang salah — sampai datanya jadi jutaan baris.

Lambat di skala besar

WAKTU PROSES QUERY DI BACKEND

JUMLAH PRODUK	KEYWORD SEARCH + FACET
100.000	~0,3 detik
1.000.000	~3,2 detik 🐌

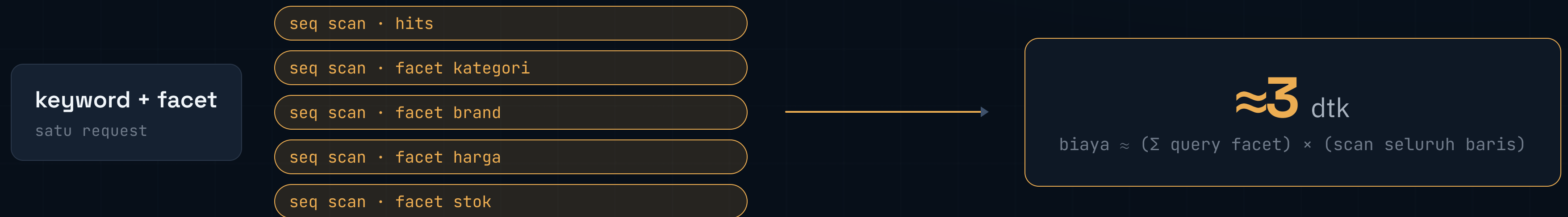
- Makin banyak data → makin lambat, mendekati linear lalu memburuk.

3 detik

Satu kali ketik pencarian di 1 juta data — **fatal untuk UX**. Dan ini baru 1 juta — bayangkan 10 juta.

Kenapa bisa selambat itu?

- `LIKE '%kata%'` (leading wildcard) → index tidak bisa dipakai → **sequential scan** seluruh tabel.
- **Facet** = beberapa query agregasi `GROUP BY`, masing-masing **full-scan** tersendiri.



Database OLTP dioptimalkan untuk transaksi tulis yang ternormalisasi – bukan untuk full-text search + agregasi berat. Tapi sebelum buru-buru ganti arsitektur: tahan dulu, coba optimalkan PostgreSQL-nya.

Tahan dulu — optimalkan database, **BARU** CQRS

1 Optimalkan database dulu — index, query, skema.

2 Ukur ulang. Buktikan dengan angka, bukan asumsi.

3 Hanya jika sudah **mentok** → pertimbangkan CQRS.

! CQRS itu **mahal**: +Kafka, +Debezium, +OpenSearch, +eventual consistency, dua datastore untuk dijaga & dimonitor.

Lompat ke CQRS karena **butuh**, bukan karena tren.

Index trigram — pg_trgm

Masalah keyword: `LIKE '%kata%'` (leading wildcard) → B-tree tak terpakai → seq scan. Solusinya ada **di dalam PostgreSQL sendiri**.

```
CREATE EXTENSION pg_trgm; CREATE INDEX idx_products_name_trgm ON products USING GIN (LOWER(name) gin_trgm_ops); CREATE INDEX idx_products_description_trgm ON products USING GIN (LOWER(description) gin_trgm_ops);
```

- **Trigram = n-gram 3 huruf**

`"mouse"` → `mou`, `ous`, `use` — disimpan di **GIN index**. Konsep yang persis sama dengan analyzer n-gram OpenSearch.

- **Nol perubahan kode**

Query `LIKE '%..%'` yang **sudah ada otomatis** memakai index. Cukup satu migration SQL — PostgreSQL punya "search engine mini" di dalamnya.

Satu index, dan search murni melesat

KEYWORD (MATCH)	PG NAIF (SEQ SCAN)	PG + TRIGRAM	SPEEDUP
titanium (874)	1.033 ms	13 ms	~79x
handcrafted (39rb)	869 ms	235 ms	~4x
rubber (80rb)	822 ms	291 ms	~3x
ergonomic (135rb)	1.118 ms	544 ms	~2x

KEYWORD SELEKTIF

~1 dtk → 13 ms — hampir secepat search engine

BUKTI SEQ SCAN

PG naif **konstan ~0,8–1,1 dtk** apa pun keyword-nya

Pelajaran besar pertama: jangan remehkan optimasi DB. Kalau kebutuhanmu hanya keyword search selektif — selesai di sini, tak perlu CQRS.

Tapi optimasi punya **batas**

✓ **Keyword search: selesai.**
pg_trgm sudah menjinakkannya

✗ **Facet tetap berat.**
titanium +facet → 357 ms • ergonomic → 1.759 ms

✗ **Skala & beban OLTP: tetap.**
GIN index ~157 MB • read tarik-menarik dgn write

- **Facet OpenSearch nyaris gratis**

OpenSearch + facet: **12–29 ms** — satu pass agregasi. Di PG, facet-lah biang lambatnya.

Di sinilah optimasi habis — dan **CQRS baru benar-benar dibutuhkan.**

Satu model untuk dua kebutuhan

WRITE

Normalisasi, konsistensi, transaksi

OLTP

READ / SEARCH

Full-text, agregasi facet, kecepatan baca

OLAP-ish

Keduanya **tarik-menarik** dalam satu model data yang sama.

Optimasi untuk write $\neq$ optimasi
untuk read.

Command Query Responsibility Segregation

- Pisahkan **model tulis (Command)** dari **model baca (Query)**.
- Tiap sisi dioptimalkan **independen**, pakai teknologi yang paling cocok.



Data ditulis ke satu model, **dibaca dari model lain** yang sudah dioptimalkan untuk baca. CQRS bukan teknologi – tapi pola.

Tidak ada makan siang gratis

+ Manfaat

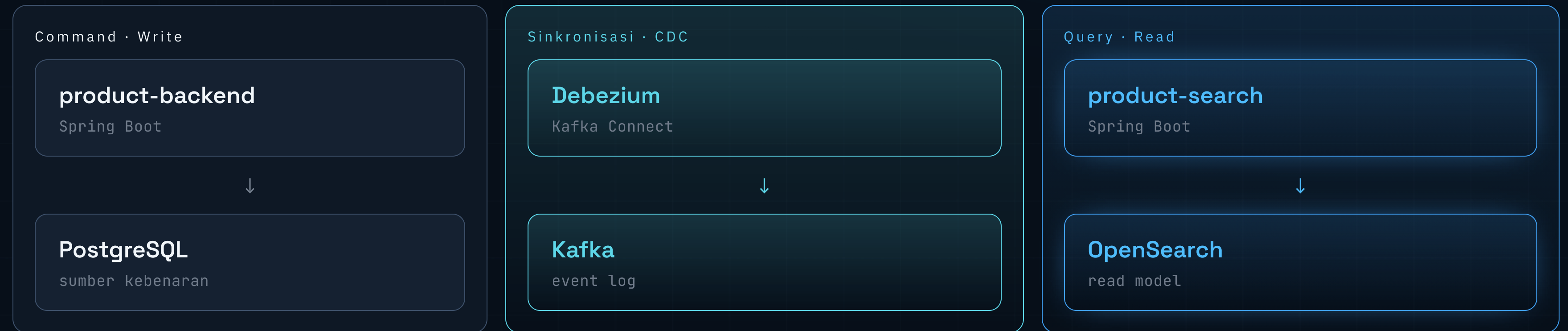
- Read model pakai **engine khusus** (search engine) → baca super cepat.
- Write model tetap **bersih & ternormalisasi**.
- Skala read & write bisa **independen**.

– Konsekuensi

- **Eventual consistency** — read model menyusul write, tidak instan.
- Lebih banyak komponen → **kompleksitas operasional** naik.

Menukar "konsistensi langsung & kesederhanaan" dengan "**kecepatan baca & skalabilitas**". Untuk search, hampir selalu worth it.

Peta besar sistem



PG → WAL logical → Debezium

Kafka → product-search → index OpenSearch

Frontend (nginx + SPA) memanggil `/api/products` ke backend, atau `/api/products/_search` ke product-search – bisa memilih sumber query.

Data write sampai ke read model — tanpa backend tahu

- `product-backend` → PostgreSQL
INSERT / UPDATE produk (`commit`)
- PostgreSQL → Debezium
perubahan via WAL (`logical`)
- Debezium → Kafka
publish event (`topik produk / brand / kategori`)
- Kafka → `product-search`
consume event
- `product-search` → OpenSearch
index / update dokumen → bisa dicari ~1 dtk kemudian

- **Membaca WAL, bukan polling**

Debezium membaca **WAL** PostgreSQL — bukan polling, bukan dual-write.

- **Penulis tak berubah**

`product-backend` tetap menulis ke PostgreSQL seperti biasa.
Write tak peduli ada berapa read model.

CDC adalah "kabel" CQRS — menangkap SEMUA perubahan tanpa mengubah aplikasi penulis.

Kenapa OpenSearch?

ASPEK	POSTGRESQL + LIKE	OPENSEARCH
Keyword substring	seq scan	terindeks
Facet + count	banyak GROUP BY	1 query agregasi
Waktu @ 1 juta	~3 detik	puluhan ms

- **Full-text & substring** cepat karena terindeks — bukan sequential scan.
- **Facet = aggregations native** → hits + semua facet dalam satu query.
- **Relevansi**, typo-tolerance, dan skala horizontal.

n-gram = LIKE tapi cepat

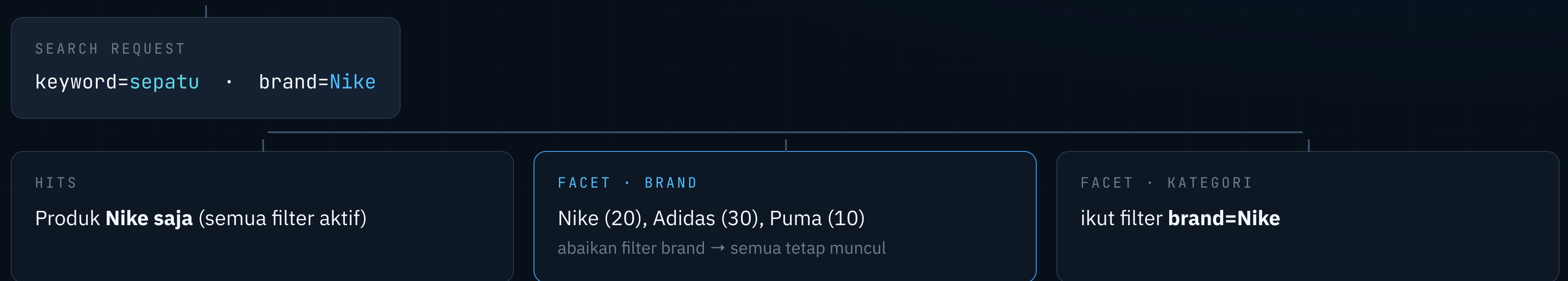
- Field `name` / `description` di-index dengan **analyzer n-gram** (2–3 huruf).
- Query memakai `match_phrase` → n-gram harus **berurutan** → cocok substring panjang berapa pun, tapi **terindeks** → **cepat**.
- Prinsip yang sama dengan `pg_trgm` (trigram) di PostgreSQL.

`"wireless"` → `[wi, wir, ire, rel, ele, les, ess, ...]` `match_phrase`: n-gram harus bersebelahan → substring `"wireless"` ketemu

Hasil demo: total hasil identik dengan **LIKE** PostgreSQL — parity terverifikasi, tanpa false positive — tapi jauh lebih cepat.

Facet drill-down

"Pilih satu brand, brand lain jangan hilang." Satu request → hits + semua facet.



Tiap facet dihitung dengan semua filter aktif **KECUALI filter dimensinya sendiri** — `post_filter` + filter aggregation per dimensi. Di PostgreSQL butuh banyak query terpisah; di OpenSearch jadi satu query elegan.

Eventual consistency



- Cocok untuk search katalog

Lag ~1 detik **sangat dapat diterima**. Tulis di backend → ~1 detik kemudian muncul di `/_search`.

- Butuh konsistensi kuat?

Untuk data seperti **saldo** → tetap baca dari write model. Pilih per use-case.

Tiga engine berdampingan

Frontend punya **toggle 3 engine** — menelusuri seluruh perjalanan optimasi. Format respons identik → tinggal tukar URL.

GET /api/products

PostgreSQL naif

~850 ms ⚠ lambat

GET /api/products?engine=trigram

PostgreSQL + trigram

~11 ms (selektif)

GET /api/products/_search

OpenSearch

~15 ms + facet murah

Param `facet=true/false` memisahkan biaya `search` murni dari `facet`. Halaman `/admin/products`: pilih engine satu klik untuk banding langsung.

Trigram menjinakkan, OpenSearch stabil

KEYWORD (MATCH)	PG NAIF	PG + TRIGRAM	OPENSEARCH
titanium (874)	1.033 ms	13 ms	13 ms
handcrafted (39rb)	869 ms	235 ms	19 ms
rubber (80rb)	822 ms	291 ms	24 ms
ergonomic (135rb)	1.118 ms	544 ms	27 ms

Trigram hebat untuk keyword selektif (titanium 1.033 → 13 ms), tapi makin umum keyword-nya makin kecil untungnya – OpenSearch tetap stabil belasan ms.

⚠ Metodologi: OpenSearch di-warmup khusus (6× semua keyword × mode) sebelum diukur; PG & trigram pakai warmup umum 3×. Semua lewat make / bench.py.

Facet nyaris gratis di OpenSearch

KEYWORD (+FACET)	PG NAIF	+ TRIGRAM	OPENSEARCH
titanium	3.352 ms	357 ms	12 ms
ergonomic	3.121 ms	1.759 ms	29 ms

RATA-RATA LINTAS KEYWORD			
ENGINE	SEARCH MURNI	+ FACET	SPEEDUP
PG naif	960 ms	3.230 ms	1×
PG + trigram	271 ms	1.102 ms	~3,5×
OpenSearch	21 ms	21 ms	~46×

OpenSearch: search & +facet sama-sama ~21 ms (satu pass agregasi).
Di PG facet berlipat → titik mentok optimasi.

Hasil sama, jauh lebih cepat

- Parity

Total hasil pencarian PostgreSQL == **OpenSearch** (terverifikasi banyak keyword). Bukan "hasil beda", tapi "hasil sama, jauh lebih cepat".

- Kecepatan

Keyword + facet @ 1 juta → **~3 dtk (PG)** vs **puluhan ms (OS)**.

- Skalabilitas

OpenSearch relatif **datar** terhadap penambahan data; PostgreSQL **LIKE** makin lama makin lambat.

Tekankan parity dulu — baru kecepatan. Tanpa parity, orang skeptis "ah mungkin hasilnya beda makanya cepat".

Komponen sistem

KOMPONEN	TEKNOLOGI	PERAN
product-backend	Java 25 · Spring Boot 4 · PostgreSQL	Write model + CRUD
Debezium + Kafka	Kafka Connect (CDC) · Kafka (KRaft)	Sinkronisasi write → read
product-search	Spring Boot 4 · Spring Kafka	Projector + Search API
OpenSearch	OpenSearch 3	Read model (search + facet)
product-frontend	SvelteKit · nginx	UI + toggle PG / OS
product-faker	Bun	Generator data demo

► Semua dijalankan via Docker / Podman Compose

Obat untuk masalah spesifik — bukan default

✓ Cocok bila

- Beban **baca jauh lebih berat** / kompleks daripada tulis — search, analitik, dashboard.
- Butuh model baca yang **beda jauh** dari model tulis.
- **Eventual consistency** dapat diterima.

✗ Hindari / tunda bila

- Data kecil & query sederhana — **over-engineering**.
- Butuh **konsistensi kuat** seketika.
- Tim belum siap dengan kompleksitas operasional (Kafka, CDC, dll).

Optimalkan dulu yang ada; pakai CQRS saat mentok

- 1 **Masalah** — LIKE + facet di PostgreSQL tidak skala (≈ 3 dtk @ 1 juta).
- 2 **Optimalkan DB dulu** — pg_trgm menurunkan keyword search 850 ms $\rightarrow$ 11 ms tanpa ubah kode.
- 3 **Batas optimasi** — trigram tak menyelesaikan facet & beban OLTP. Di sinilah CQRS beralasan.
- 4 **CQRS** — pisahkan write (PostgreSQL) dari read (OpenSearch); jembatan CDC (Debezium $\rightarrow$ Kafka).
- 5 **Hasil** — hasil sama, kecepatan puluhan ms (search + facet) vs detik.
- 6 **Harga** — eventual consistency + kompleksitas. Pilih sesuai kebutuhan.

— TERIMA KASIH • Q&A

Database yang tepat untuk pekerjaan yang tepat.

Optimalkan dulu yang ada — pakai CQRS saat memang mentok,
tanpa mengorbankan sumber kebenaran.

► Repo: [ProgrammerZamanNow/cqrs-demo](#)

`podman compose up -d --build`

`make register` → buka :3000